```
> # ============================================================
  # TauTimp -- Thermal Particle Wave Equation & Probability PDE
  # Maple transcript (paired with TauTimp_ThermalParticle.md)
  # Author: Viorel
  # ============================================================

  restart;
  with(plots):
  with(plottools):
  interface(displayprecision = 10):

  # ------------------------------------------------------------
  # SECTION 1. Physical constants and parameters
  # ------------------------------------------------------------
  hbar  := 1.054571817e-34:        # J*s
  me    := 9.1093837e-31:          # kg
  eV    := 1.602176634e-19:        # J
  c0    := 2.99792458e8:           # m/s
  R     := 4.235518529e22:              # m  (TauTimp cosmological
  radius)
  lqn   := 1/2:                    # spin magnetic moment quantum
  number
  llp1  := lqn*(lqn+1);            # l(l+1) = 3/4

  printf("R          = %.6e m\n",   evalf(R)):
  printf("m_e        = %.6e kg\n",  evalf(me)):
  printf("hbar       = %.6e J*s\n", evalf(hbar)):
  printf("l(l+1)     = %.6f\n",     evalf(llp1)):

  # ------------------------------------------------------------
  # SECTION 2. The governing PDE (symbolic)
  # ------------------------------------------------------------
  # (2m/hbar^2)(E - l(l+1) hbar^2/(2 m r^2)) f + (R/r^2) d^2(r f)
  /dr^2
  #                                          = (1/V^2) d^2 f/dt^2
  PDE_full := (2*m/hbar^2)*(E - lqn*(lqn+1)*hbar^2/(2*m*r^2))*F(r,
  t)
            + (R/r^2)*diff(r*F(r,t), r$2)
            - (1/V^2)*diff(F(r,t), t$2) = 0;

  print("Full TauTimp thermal-particle PDE:"):
  print(PDE_full):
```

```
# ------------------------------------------------------------
# SECTION 3. Separation of variables F(r,t) = g(r) * exp(-I omega
t)
# ------------------------------------------------------------
# Substitute h(r) = r * g(r) and use d^2(r g)/dr^2 = 2 g' + r g''
# Result: R h''(r) + ( 2 m E/hbar^2 + omega^2/V^2 - l(l+1)/r^2 )
h(r) = 0

ODE_h := R*diff(H(r), r$2)
       + ( 2*m*Ev/hbar^2 + omega^2/V^2 - lqn*(lqn+1)/r^2 )*H(r) =
0;

print("Reduced radial ODE (h(r) = r*g(r)):"):
print(ODE_h):

# Because l(l+1)/R is cosmologically tiny, the equation reduces
to
#   h''(r) + kappa^2 h(r) = 0 ,  with kappa^2 = K/R ,
# K := 2 m E/hbar^2 + omega^2/V^2

kappa := 'kappa':
H_sol := A*sin(kappa*r) + B*cos(kappa*r);
print("Reduced free-radial solution:"):
print( H(r) = H_sol ):

# Regularity at r=0 of g(r)=h(r)/r forces B=0
g_sol := A*sin(kappa*r)/r;
print("Radial eigenfunction g(r):"):
print( g(r) = g_sol ):

# ------------------------------------------------------------
# SECTION 4. Quantization from f(R,t) = 0
# ------------------------------------------------------------
# sin(kappa_n R) = 0  =>  kappa_n = n*Pi/R
# Energy:  E_n = hbar^2 n^2 Pi^2 / (2 m R)

E_of_n := unapply( hbar^2 * n^2 * Pi^2 / (2 * me * R), n ):

printf("\n--- Energy spectrum E_n (TauTimp thermal modes) ---\n")
:
for k from 1 to 6 do
  En_J  := evalf( E_of_n(k) ):
```

```
  En_eV := evalf( En_J / eV ):
  printf("n=%d :  E_n = %.6e J  =  %.6e eV\n", k, En_J, En_eV):
end do:

# Quantization wavevector and wavelength
printf("\n--- kappa_n and wavelength lambda_n = 2*Pi/kappa_n ---
\n"):
for k from 1 to 6 do
  kn  := evalf( k*Pi/R ):
  lan := evalf( 2*Pi/kn ):
  printf("n=%d :  kappa_n = %.6e 1/m   lambda_n = %.6e m\n", k,
kn, lan):
end do:

# ------------------------------------------------------------
# SECTION 5. Radial eigenfunction plot
# ------------------------------------------------------------
# Plot g_n(r) = sin(n*Pi*r/R)/r on r in [R/1000, R] for n = 1,2,3
# Use dimensionless x = r/R for clarity; g shown in units of 1/R.

g_dimless := (nn, x) -> sin(nn*Pi*x)/x:    # actually sin(n*Pi*
r/R)/(r) * R --> shown vs x=r/R

plot_radial := plot(
  [g_dimless(1, x), g_dimless(2, x), g_dimless(3, x)],
  x = 0.01 .. 1,
  color  = [red, blue, "DarkGreen"],
  thickness = 2,
  labels = ["r / R", "g_n(r) * R"],
  title  = "TauTimp thermal radial eigenfunctions n=1,2,3",
  legend = ["n = 1", "n = 2", "n = 3"]
):

# ------------------------------------------------------------
# SECTION 6. Probability-density PDE and Gaussian envelope
# ------------------------------------------------------------
# Equation:  d^2 P/dt^2 - d^2 (P V^2)/dr^2 = V^2 P / R
# For V = const = Vbar:
#   d^2 P/dt^2 - Vbar^2 d^2 P/dr^2 = Vbar^2 P / R

PDE_P := diff(P(r,t), t$2) - Vb^2*diff(P(r,t), r$2) = Vb^2*P(r,t)
/R;
```

```
print("Probability-density PDE (constant V):"):
print(PDE_P):

# Gaussian ansatz: P(r,t) = N(t) * exp( -(r-r0)^2 / (2 sigma(t)
^2) )
P_ansatz := N(t) * exp( -(r-r0)^2 / (2*sigma(t)^2) );
print("Gaussian ansatz:"):
print( P(r,t) = P_ansatz ):

# Match coefficients => ODE system:
#   (r-r0)^4 :  d/dt sigma = +/- Vbar
#   (r-r0)^2 :  d/dt log(N) = 2 Vbar / sigma
#   (r-r0)^0 :  d^2/dt^2(N)/N + Vbar^2/sigma^2 = Vbar^2 / R

# General solution with sigma(0)=sigma0, N(0)=N0:
sigma_t := (sigma0, Vb, t) -> sigma0 + Vb*t:
N_t     := (N0, sigma0, Vb, t) -> N0*( 1 + Vb*t/sigma0 )^2:

# Consistency at the validity point: 3 Vbar^2/sigma^2 = Vbar^2/R
#   =>  sigma0 = sqrt(3 R)
sigma0_validity := sqrt(3*R):
printf("\n--- Envelope consistency point ---\n"):
printf("sigma0_validity = sqrt(3R) = %.6e m\n", evalf
(sigma0_validity)):
printf("    (i.e. the natural cosmological width that closes the
ODE)\n"):

# Check boundary condition P(2R, 0) = 0 in the asymptotic sense
# (Gaussian is never identically zero; we report the suppression)
suppression := evalf( exp( -(2*R)^2/(2*(sigma0_validity)^2) ) ):
printf("\n--- Boundary condition P(2R,0) ~ exp(-2R/3) ---\n"):
printf("Suppression factor exp(-(2R)^2/(2 sigma0^2)) = exp(-2R/3)
= %.6e\n",
        suppression):
printf("    => effectively zero on any physical scale\n"):

# ------------------------------------------------------------
# SECTION 7. Numerical envelope evolution -- normalized units
# ------------------------------------------------------------
# To plot in finite range, normalize: tau = Vb*t/sigma0,  xi =
(r-r0)/sigma0
# Then  P/N0 = (1+tau)^2 * exp( -xi^2 / (2 (1+tau)^2) )
```

```
# This shows the Gaussian envelope spreading linearly in tau.

P_norm := (xi, tau) -> (1+tau)^2 * exp( -xi^2 / (2*(1+tau)^2) ):

# Snapshot plots at tau = 0, 0.5, 1.0, 2.0
plot_envelope := plot(
  [ P_norm(xi, 0), P_norm(xi, 0.5), P_norm(xi, 1.0), P_norm(xi,
2.0) ],
  xi = -8 .. 8,
  color  = [red, "Orange", blue, "DarkGreen"],
  thickness = 2,
  labels = ["xi = (r-r0)/sigma0", "P / N0"],
  title  = "Gaussian envelope evolution -- TauTimp probability
density",
  legend = ["tau = 0", "tau = 0.5", "tau = 1.0", "tau = 2.0"]
):

# ------------------------------------------------------------
# SECTION 8. 3D surface  P_norm(xi, tau)
# ------------------------------------------------------------
plot_surface := plot3d(
  P_norm(xi, tau),
  xi  = -6 .. 6,
  tau =  0 .. 2,
  axes = boxed,
  labels = ["xi", "tau", "P / N0"],
  title  = "Probability density P/N0 vs. (xi, tau)",
  colorscheme = ["zgradient", ["MidnightBlue", "Cyan", "Yellow",
"OrangeRed"]],
  style = surface,
  grid = [60, 40]
):

# ------------------------------------------------------------
# SECTION 9. Energy-spectrum bar chart
# ------------------------------------------------------------
En_list := [ seq( [k, evalf( E_of_n(k)/eV ) ], k = 1 .. 8 ) ]:
plot_energy := plot(
  En_list,
  style = pointline,
  symbol = solidcircle,
  symbolsize = 16,
```

```
    color = "DarkRed",
    thickness = 2,
    labels = ["n", "E_n  [eV]"],
    title  = "Thermal-particle eigenenergies E_n = hbar^2 n^2 Pi^2/
  (2 m R)"
  ):

  # ------------------------------------------------------------
  # SECTION 10. Combined panel
  # ------------------------------------------------------------
  display( Array( [ plot_radial, plot_envelope, plot_energy,
  plot_surface ] ) );

  # ------------------------------------------------------------
  # SECTION 11. Final diagnostics
  # ------------------------------------------------------------
  printf("\n=================  SUMMARY  =================\n"):
  printf("Quantization:         kappa_n = n*Pi/R\n"):
  printf("Eigenenergies:        E_n = hbar^2 n^2 Pi^2 / (2 m R)\n"):
  printf("  E_1 = %.6e eV\n", evalf( E_of_n(1)/eV ) ):
  printf("  E_2 = %.6e eV\n", evalf( E_of_n(2)/eV ) ):
  printf("  E_3 = %.6e eV\n", evalf( E_of_n(3)/eV ) ):
  printf("Envelope width:       sigma(t) = sigma0 + Vbar*t,  sigma0
  = sqrt(3R)\n"):
  printf("Envelope amplitude:   N(t) = N0 (1 + Vbar*t/sigma0)^2\n"):
  printf("Boundary:             P(2R,0) ~ exp(-2R/3) = %.3e  (=> 0)
  \n", suppression):
  printf("=============================================\n"):
```

$$llp1 := \frac{3}{4}$$

```
R           = 4.235519e+22 m
m_e         = 9.109384e-31 kg
hbar        = 1.054572e-34 J*s
l(l+1)      = 0.750000
```

$$PDE_full := 1.7983643060 \times 10^{68}\, m \left(E - \frac{4.1704564390 \times 10^{-69}}{m\, r^2} \right) F(r,t) + \frac{4.2355185290 \times 10^{22} \left(2 \frac{\partial}{\partial r} F(r,t) + r \left(\frac{\partial^2}{\partial r^2} F(r,t) \right) \right)}{r^2} - \frac{\partial^2}{\partial t^2} \frac{F(r,t)}{V^2} = 0$$

"Full TauTimp thermal-particle PDE:"

$$1.7983643060 \times 10^{68}\, m \left(E - \frac{4.1704564390 \times 10^{-69}}{m\, r^2} \right) F(r,t) + \frac{4.2355185290 \times 10^{22} \left(2 \frac{\partial}{\partial r} F(r,t) + r \left(\frac{\partial^2}{\partial r^2} F(r,t) \right) \right)}{r^2} - \frac{\frac{\partial^2}{\partial t^2} F(r,t)}{V^2} = 0$$

$$ODE_h := 4.2355185290 \times 10^{22} \frac{d^2}{dr^2} H(r) + \left(1.7983643060 \times 10^{68}\, m\, Ev + \frac{\omega^2}{V^2} - \frac{3}{4\, r^2} \right) H(r) = 0$$

"Reduced radial ODE (h(r) = r*g(r)):"

$$4.2355185290 \times 10^{22} \frac{d^2}{dr^2} H(r) + \left(1.7983643060 \times 10^{68}\, m\, Ev + \frac{\omega^2}{V^2} - \frac{3}{4\, r^2} \right) H(r) = 0$$

$$H_sol := A \sin(\kappa r) + B \cos(\kappa r)$$

"Reduced free-radial solution:"

$$H(r) = A \sin(\kappa r) + B \cos(\kappa r)$$

$$g_sol := \frac{A \sin(\kappa r)}{r}$$

"Radial eigenfunction g(r):"

$$g(r) = \frac{A \sin(\kappa r)}{r}$$

```
--- Energy spectrum E_n (TauTimp thermal modes) ---
n=1 :  E_n = 1.422416e-60 J  =  8.878020e-42 eV
n=2 :  E_n = 5.689662e-60 J  =  3.551208e-41 eV
n=3 :  E_n = 1.280174e-59 J  =  7.990218e-41 eV
n=4 :  E_n = 2.275865e-59 J  =  1.420483e-40 eV
n=5 :  E_n = 3.556039e-59 J  =  2.219505e-40 eV
n=6 :  E_n = 5.120696e-59 J  =  3.196087e-40 eV

--- kappa_n and wavelength lambda_n = 2*Pi/kappa_n ---
n=1 :  kappa_n = 7.417256e-23 1/m   lambda_n = 8.471037e+22 m
n=2 :  kappa_n = 1.483451e-22 1/m   lambda_n = 4.235519e+22 m
n=3 :  kappa_n = 2.225177e-22 1/m   lambda_n = 2.823679e+22 m
n=4 :  kappa_n = 2.966903e-22 1/m   lambda_n = 2.117759e+22 m
n=5 :  kappa_n = 3.708628e-22 1/m   lambda_n = 1.694207e+22 m
n=6 :  kappa_n = 4.450354e-22 1/m   lambda_n = 1.411840e+22 m
```

$$PDE_P := \frac{\partial^2}{\partial t^2} P(r,t) - Vb^2 \left(\frac{\partial^2}{\partial r^2} P(r,t) \right) = 2.3609860120 \times 10^{-23} Vb^2 P(r,t)$$

"Probability-density PDE (constant V):"

$$\frac{\partial^2}{\partial t^2} P(r,t) - Vb^2 \left(\frac{\partial^2}{\partial r^2} P(r,t) \right) = 2.3609860120 \times 10^{-23} Vb^2 P(r,t)$$

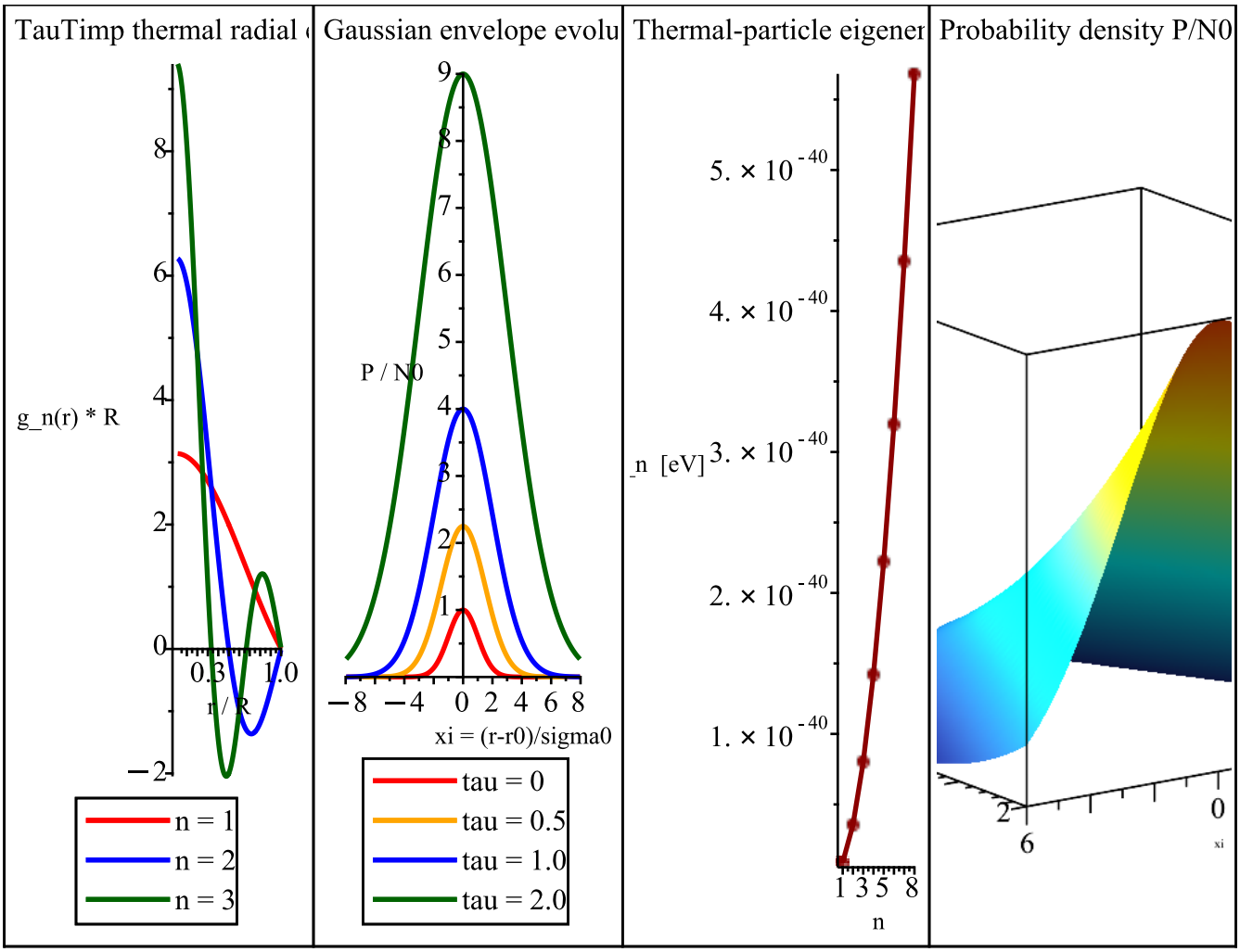
$$P_ansatz := N(t)\, e^{-\frac{(r-r0)^2}{2\,\sigma(t)^2}}$$

"Gaussian ansatz:"

$$P(r,t) = N(t)\, e^{-\frac{(r-r0)^2}{2\,\sigma(t)^2}}$$

```
--- Envelope consistency point ---
sigma0_validity = sqrt(3R) = 3.564626e+11 m
    (i.e. the natural cosmological width that closes the ODE)

--- Boundary condition P(2R,0) ~ exp(-2R/3) ---
Suppression factor exp(-(2R)^2/(2 sigma0^2)) = exp(-2R/3) =
0.000000e+00
    => effectively zero on any physical scale
```




```
================   SUMMARY   ================
Quantization:          kappa_n = n*Pi/R
Eigenenergies:         E_n = hbar^2 n^2 Pi^2 / (2 m R)
  E_1 = 8.878020e-42 eV
  E_2 = 3.551208e-41 eV
  E_3 = 7.990218e-41 eV
Envelope width:        sigma(t) = sigma0 + Vbar*t,  sigma0 = sqrt(3R)
Envelope amplitude:    N(t) = N0 (1 + Vbar*t/sigma0)^2
Boundary:              P(2R,0) ~ exp(-2R/3) = 0.000e+00   (=> 0)
============================================
```